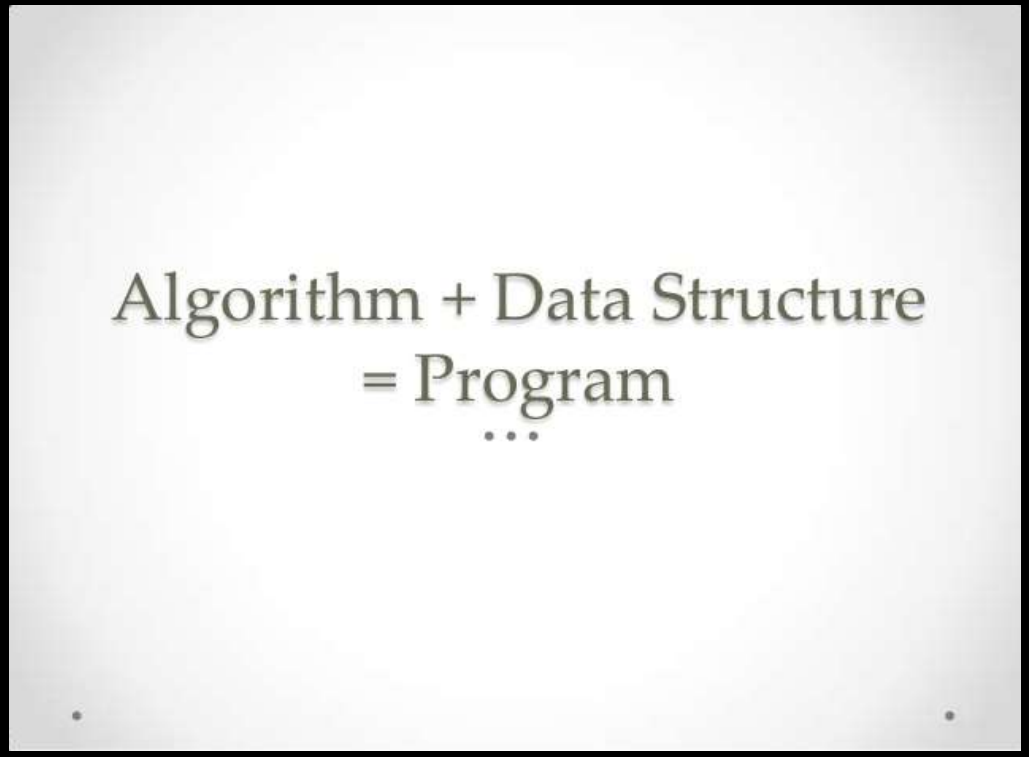
ALGORITHM COMPLEXITY AND MAIN CONCEPTS

- Data structures and algorithms are the foundation of computer programming
- Algorithmic thinking, problem solving and data structures are vital for software engineers
- Computational complexity is important for algorithm design and efficient programming

Data structures and algorithms are the fundamentals of programming. In order to become a good developer it is essential to master the basic data structures and algorithms and learn to apply them in the right way.



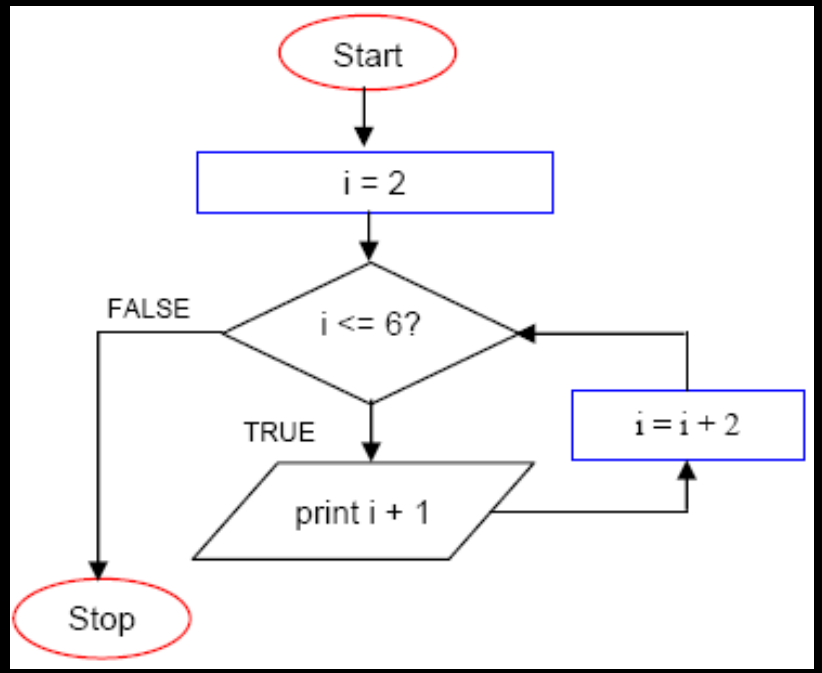
WHAT IS AN ALGORITHM?

THE ENGLISH WORD "ALGORITHM" DERIVES FROM THE LATIN FORM OF AL-AL-KHWARIZMI'S NAME. HE DEVELOPED THE CONCEPT OF AN ALGORITHM ALGORITHM IN MATHEMATICS, AND IS THUS SOMETIMES GIVEN THE TITLE OF "GRANDFATHER OF COMPUTER SCIENCE".

- *MUHAMMAD IBN MUSA AL-KHWARIZMI (APPROX 800-847 CE)*
- *BORN AROUND 800 CE IN KHWARIZM, NOW IN UZBEKISTAN.*
- *AL-KHWARIZMI LIVED IN BAGHDAD, WHERE HE WORKED AT THE "HOUSE OF WISDOM" (DAR AL-HIKMA) UNDER THE CALIPH HARUN AL-RASHID.*

WHAT IS AN ALGORITHM?

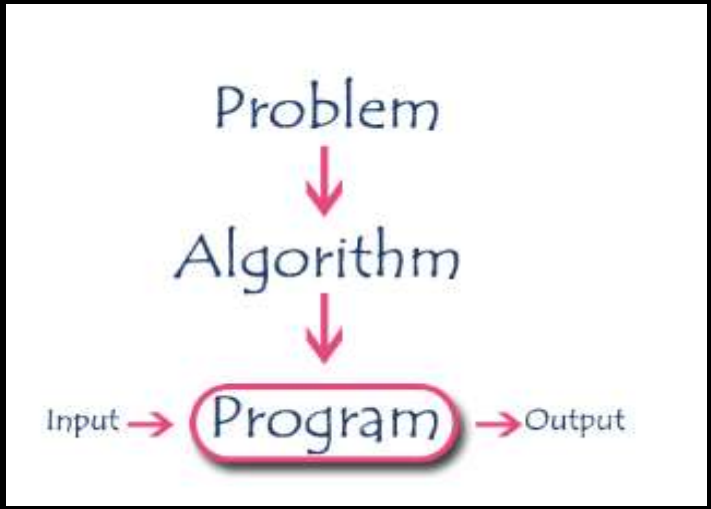
An algorithm is a finite set of instructions or logic, written in order, to accomplish a certain predefined task. Algorithm is not the complete code or program, it is just the core logic(solution) of a problem, which can be expressed either as an informal high level description as pseudocode or using a flowchart.



WHILE loop

- Do the loop body if the condition is true.
- Example: Get the sum of 1, 2, 3, …, 100.
  - Algorithm:
    - Set the number = 1
    - Set the total = 0
    - While (number <= 100)
      - total = total + number
      - number = number + 1
    - End While
    - Display total

ALGORITHM SPECIFICATIONS



EVERY ALGORITHM MUST SATISFY THE FOLLOWING SPECIFICATIONS...

INPUT - EVERY ALGORITHM MUST TAKE ZERO OR MORE NUMBER OF INPUT VALUES FROM EXTERNAL.

OUTPUT - EVERY ALGORITHM MUST PRODUCE AN OUTPUT AS RESULT.

DEFINITENESS - EVERY STATEMENT/INSTRUCTION IN AN ALGORITHM MUST BE CLEAR AND UNAMBIGUOUS (ONLY ONE INTERPRETATION)

FINITENESS - FOR ALL DIFFERENT CASES, THE ALGORITHM MUST PRODUCE RESULT WITHIN A FINITE NUMBER OF STEPS.

EFFECTIVENESS - EVERY INSTRUCTION MUST BE BASIC ENOUGH TO BE CARRIED OUT AND IT ALSO MUST BE FEASIBLE.

Example of Algorithm

Let us consider the following problem for finding the largest value in a given list of values.

Problem Statement : Find the largest number in the given list of numbers
Input : A list of positive integer numbers. (List must contain at least one number)
Output : The largest number in the given list of positive integer numbers
Consider the given list of numbers as 'L' (input), and the largest number as 'max' (Output).

Algorithm

Step 1: Define a variable **'max'** and initialize with **'0'**.

Step 2: Compare first number (say **'x'**) in the list **'L'** with **'max'**, if **'x'** is larger than **'max'**, set **'max'** to **'x'**.

Step 3: Repeat step 2 for all numbers in the list **'L'**.

Step 4: Display the value of **'max'** as a result.

```
int findMax(L)
{
   int max = 0,i;
   for(i=0; i < listSize; i++)
   {
      if(L[i] > max)
         max = L[i];
   }
   return max;
}
```

RECURSIVE ALGORITHM

- The function which calls by itself is called as **Direct Recursive function** (or Recursive function)
- The function which calls a function and that function calls its called function is called **Indirect Recursive function** (or Recursive function)

PERFORMANCE ANALYSIS

What is Performance Analysis of an algorithm?

Performance of an algorithm is a process of making evaluative judgement about algorithms.

Performance of an algorithm means predicting the resources which are required to an algorithm to perform its task.

An algorithm is said to be efficient and fast, if it takes less time to execute and consumes less memory space. The performance of an algorithm is measured on the basis of following properties :

- **Space Complexity**
- **Time Complexity**

SPACE COMPLEXITY

Its the amount of memory space required by the algorithm, during the course of its execution. Space complexity must be taken seriously for multi-user systems and in situations where limited memory is available.

An algorithm generally requires space for following components :

- **Instruction Space :** Its the space required to store the executable version of the program. This space is fixed, but varies depending upon the number of lines of code in the program.
- **Data Space :** Its the space required to store all the constants and variables value.
- **Environment Space :** Its the space required to store the environment information needed to resume the suspended function.

Constant Space Complexity

```
int square(int a)
{
      return a*a;
}
```

If any algorithm requires a fixed amount of space for all input values then that space complexity is said to be Constant Space Complexity

- # Linear Space Complexity

```
int sum(int A[], int n)
{
   int sum = 0, i;
   for(i = 0; i < n; i++)
      sum = sum + A[i];
   return sum;
}
```

If the amount of space required by an algorithm is increased with the increase of input value, then that space complexity is said to be Linear Space Complexity

TIME COMPLEXITY

The time complexity of an algorithm is the total amount of time required by an algorithm to complete its execution.

If any program requires fixed amount of time for all input values then its time complexity is said to be Constant Time Complexity

```
int sum(int a, int b)
{
    return a+b;
}
```

Linear Time Complexity

If the amount of time required by an algorithm is increased with the increase of input value then that time complexity is said to be Linear Time Complexity

```
int sum(int A[], int n)
{
    int sum = 0, i;
    for(i = 0; i < n; i++)
        sum = sum + A[i];
    return sum;
}
```

int sumOfList(int A[ ], int n)	Cost Time require for line (Units)	Repeatation No. of Times Executed	Total Total Time required in worst case
{			
int sum = 0, i;	1	1	1
for(i = 0; i < n; i++)	1 + 1 + 1	1 + (n+1) + n	2n + 2
sum = sum + A[i];	2	n	2n
return sum;	1	1	1
}			
			4n + 4 Total Time required

ASYMPTOTIC NOTATION

Asymptotic notation of an algorithm is a mathematical representation of its complexity

For example, consider the following time complexities of two algorithms...

Algorihtm 1 : 5n2 + 2n + 1

Algorihtm 2 : 10n2 + 8n + 3

Majorly, we use THREE types of Asymptotic Notations and those are as follows...

Big - Oh (O)

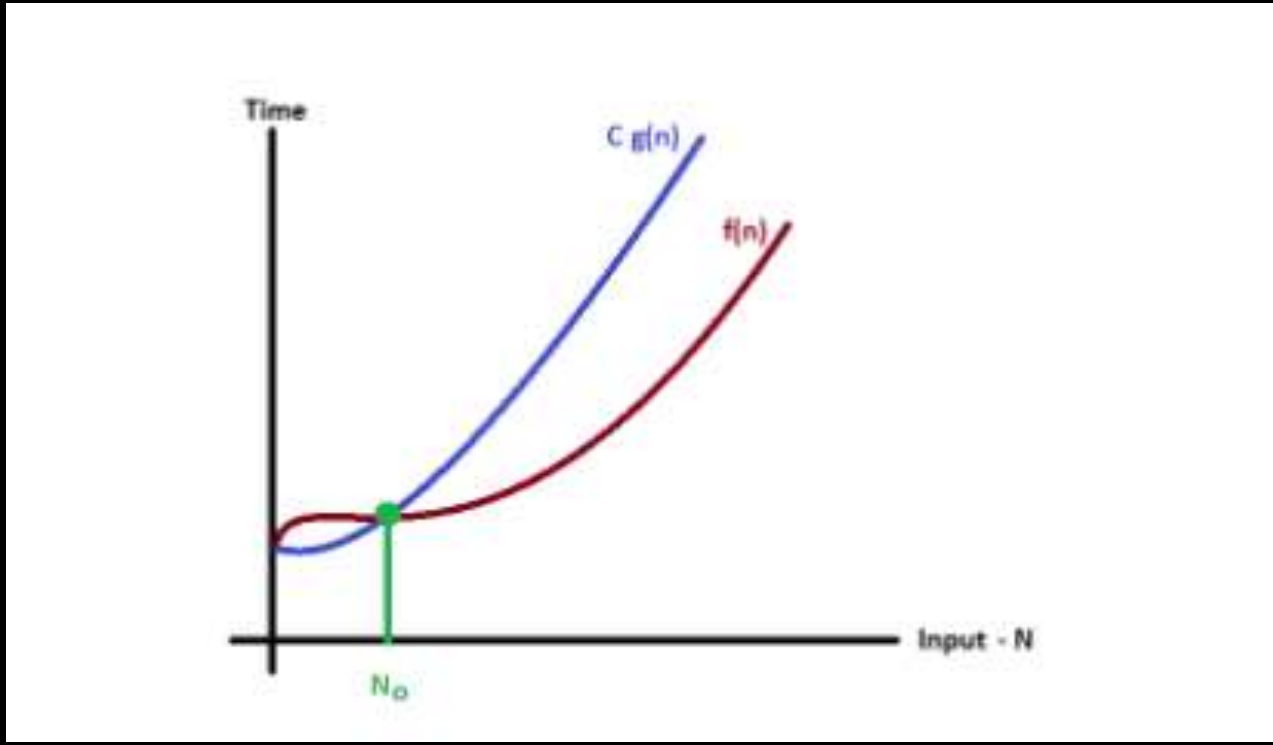
Big - Omega (Ω)

Big - Theta (Θ)

BIG - OH NOTATION (O)

Consider function f(n) the time complexity of an algorithm and g(n) is the most significant term. If f(n) <= C g(n) for all n >= n0, C > 0 and n0 >= 1. Then we can represent f(n) as O(g(n)).

$$f(n) = O(g(n))$$



HOW TO CALCULATE TIME COMPLEXITY?

*Now the most common metric for calculating time complexity is Big O notation. This removes all constant factors so that the running time can be estimated in relation to **N**, as N approaches infinity. In general you can think of it like this :*

```
statement;
```

*Above we have a single statement. Its Time Complexity will be **Constant**. The running time of the statement will not change in relation to N.*

```
for(i=0; i < N; i++)
{
  statement;
}
```

*The time complexity for the above algorithm will be **Linear**. The running time of the loop is directly proportional to N. When N doubles, so does the running time.*

```
for(i=0; i < N; i++)
{
  for(j=0; j < N;j++)
  {
    statement;
  }
}
```

*This time, the time complexity for the above code will be **Quadratic**. The running time of the two loops is proportional to the square of N. When N doubles, the running time increases by N * N.*

```
while(low <= high)
{
  mid = (low + high) / 2;
  if (target < list[mid])
    high = mid - 1;
  else if (target > list[mid])
    low = mid + 1;
  else break;
}
```

This is an algorithm to break a set of numbers into halves, to search a particular field(we will study this in detail later). Now, this algorithm will have a ***Logarithmic*** *Time Complexity. The running time of the algorithm is proportional to the number of times N can be divided by 2(N is high-low here). This is because the algorithm divides the working area in half with each iteration.*

```
void quicksort(int list[], int left, int right)
{
  int pivot = partition(list, left, right);
  quicksort(list, left, pivot - 1);
  quicksort(list, pivot + 1, right);
}
```

Taking the previous algorithm forward, above we have a small logic of Quick Sort. Now in Quick Sort, we divide the list into halves every time, but we repeat the iteration N times(where N is the size of list). Hence time complexity will be **N*log(N)**. The running time consists of N loops (iterative or recursive) that are logarithmic, thus the algorithm is a combination of linear and logarithmic.

EXAMPLE

f(n) = 3n + 2
g(n) = n

O(g(n))

f(n) <= C x g(n) C > 0 and **n0>= 1**

f(n) <= C g(n)
⇒3n + 2 <= C n

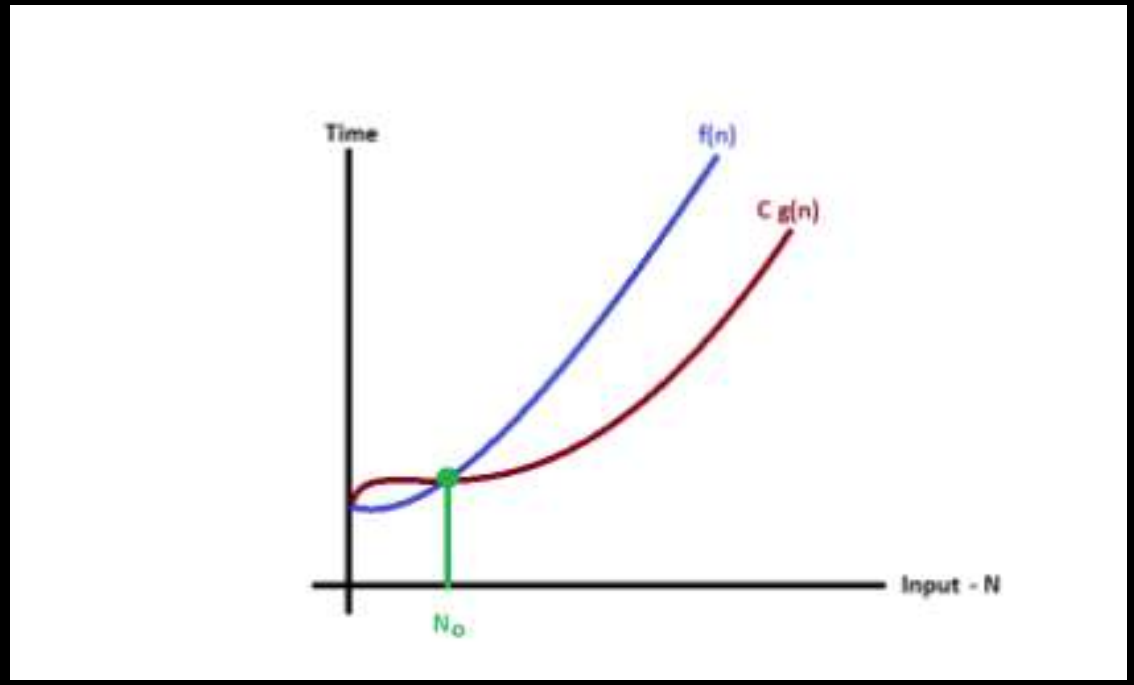
Above condition is always TRUE for all values of **C = 4** and **n >= 2**.
By using Big - Oh notation we can represent the time complexity as follows...
3n + 2 = O(n)

BIG - OMEGE NOTATION (Ω)

Big - Omega notation is used to define the **lower bound** of an algorithm in terms of Time Complexity.

Consider function f(n) the time complexity of an algorithm and g(n) is the most significant term. If f(n) >= C x g(n) for all n >= n0, C > 0 and n0 >= 1. Then we can represent f(n) as Ω(g(n)).

f(n) = Ω(g(n))



EXAMPLE

Consider the following f(n) and g(n)...
f(n) = 3n + 2
g(n) = n
If we want to represent **f(n)** as **Ω(g(n))** then it must satisfy **f(n) >= C g(n)** for all values of **C > 0** and **n0>= 1**

f(n) >= C g(n)
⇒3n + 2 <= C n

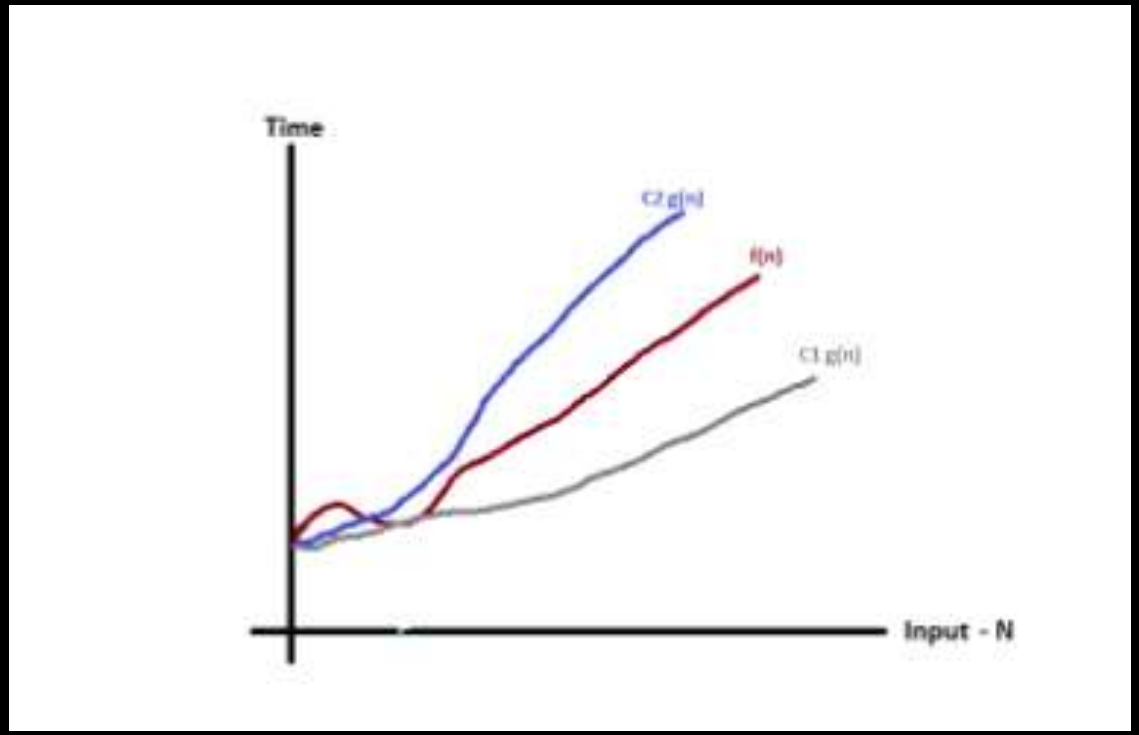
Above condition is always TRUE for all values of **C = 1** and **n >= 1**.
By using Big - Omega notation we can represent the time complexity as follows...
3n + 2 = Ω(n)

BIG - THETA NOTATION (Θ)

Big - Theta notation is used to define the **average bound** of an algorithm in terms of Time Complexity.

Consider function f(n) the time complexity of an algorithm and g(n) is the most significant term. If C1g(n) <= f(n) >= C2 g(n) for all n >= n0, C1, C2 > 0 and n0 >= 1. Then we can represent f(n) as Θ(g(n)).

$$f(n) = \Theta(g(n))$$



EXAMPLE

Consider the following f(n) and g(n)...
f(n) = 3n + 2
g(n) = n
If we want to represent **f(n)** as **Θ(g(n))** then it must satisfy **C1 g(n) <= f(n) >= C2 g (n)** for all values of **C1, C2 > 0** and **n0>= 1**

C1 g(n) <= f(n) >= ⇒C2 g(n)
C1 n <= 3n + 2 >= C2 n

Above condition is always TRUE for all values of **C1 = 1, C2 = 4** and **n >= 1**.
By using Big - Theta notation we can represent the time complexity as follows...
3n + 2 = Θ(n)

- ***NOTE :*** In general, doing something with every item in one dimension is linear, doing something with every item in two dimensions is quadratic, and dividing the working area in half is logarithmic.

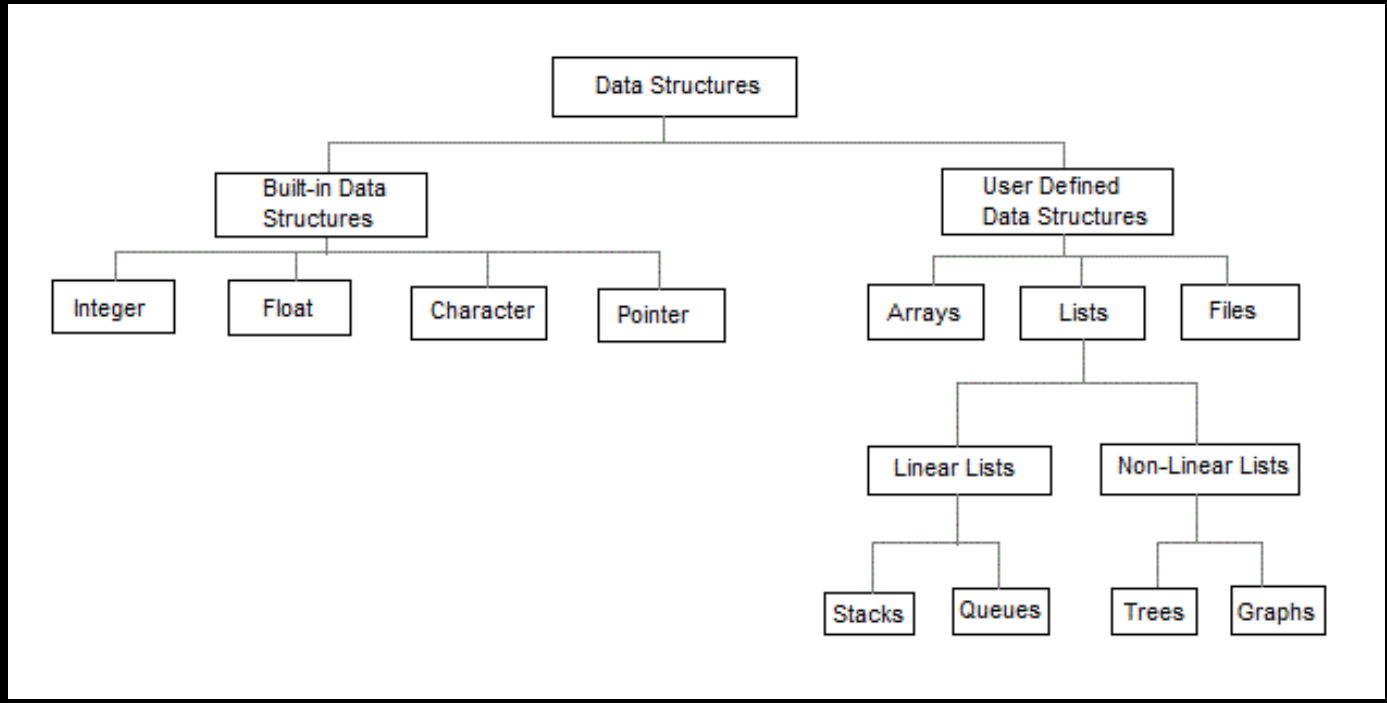
DATA STRUCTURES

- Data Structure is a way of collecting and **organizing** data in such a way that we can perform operations on these data in an effective way. Data Structures is about rendering data elements in terms of some relationship, for better organization and storage. For example, we have data player's name "Virat" and age 26. Here "Virat" is of **String** data type and 26 is of **integer** data type.
- We can organize this data as a record like **Player** record. Now we can collect and store player's records in a file or database as a data structure. For example: "Dhoni" 30, "Gambhir" 31, "Sehwag" 33
- In simple language, Data Structures are structures programmed to store ordered data, so that various operations can be performed on it easily.

BASIC TYPES OF DATA STRUCTURES

- **Primitive Data Structures :**
- Integer,
- Float,
- Boolean,
- Char etc

- **Abstract Data Structure** :
- Linked List
- Tree
- Graph
- Stack, Queue etc.



If a data structure is organizing the data in *sequential order*, then that data structure is called as Linear Data Structure.

- Arrays
- List (Linked List)
- Stack
- Queue

If a data structure is organizing the data in *random order*, then that data structure is called as Non-Linear Data Structure.

- Tree
- Graph
- Dictionaries
- Heaps
- Tries, Etc.,

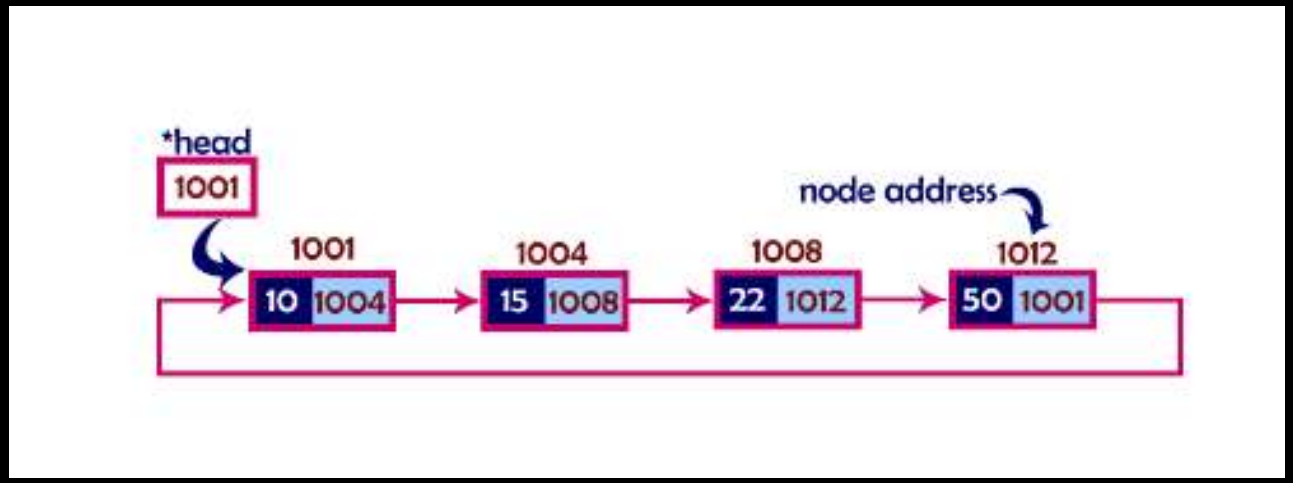
SINGLE LINKED LIST

Single linked list is a sequence of elements in which every element has link to its next element in the sequence.

☼ In a single linked list, the address of the first node is always stored in a reference node known as "front" (Some times it is also known as "head").
☼ Always next part (reference part) of the last node must be NULL.

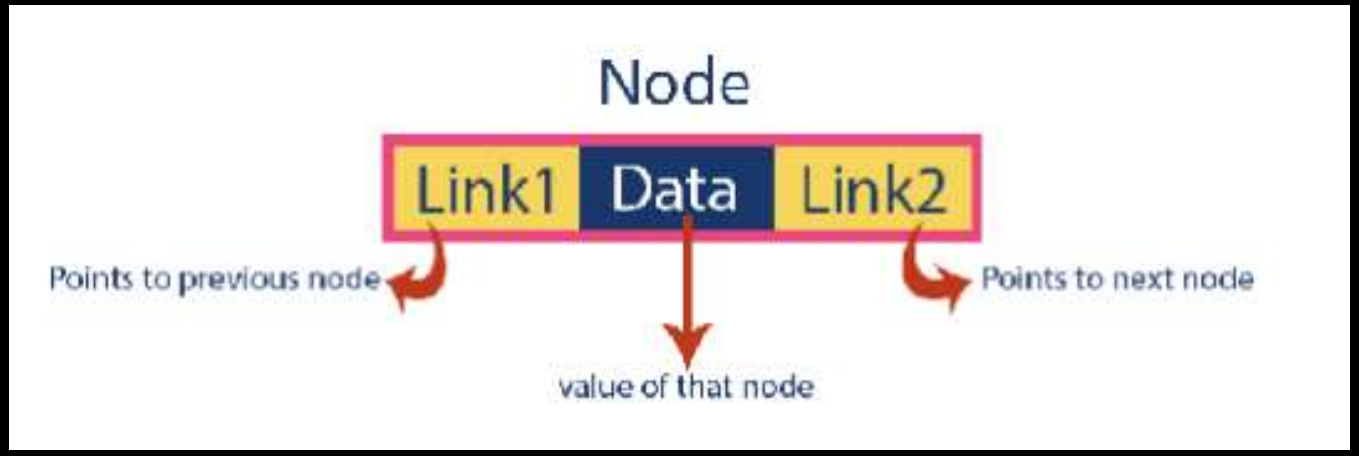
CIRCULAR LINKED LIST

Circular linked list is a sequence of elements in which every element has link to its next element in the sequence and the last element has a link to the first element in the sequence.



DOUBLE LINKED LIST

Double linked list is a sequence of elements in which every element has links to its previous element and next element in the sequence.



- In double linked list, the first node must be always pointed by **head**.
- Always the previous field of the first node must be **NULL**.
- Always the next field of the last node must be **NULL**.

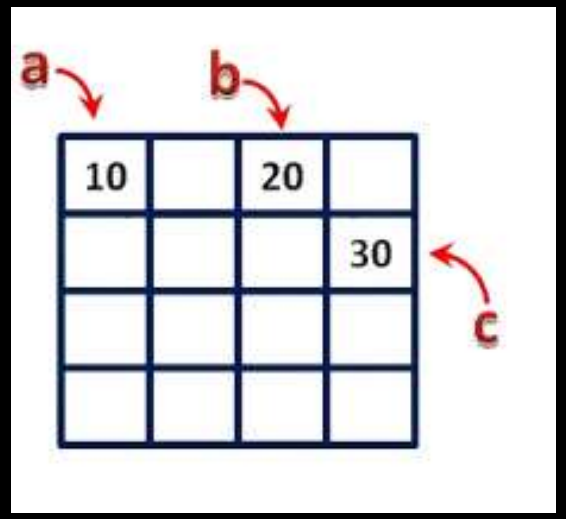
ARRAYS

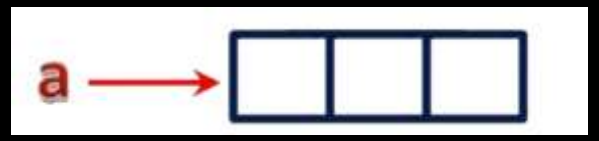
An array is a variable which can store multiple values of same data type at a time.

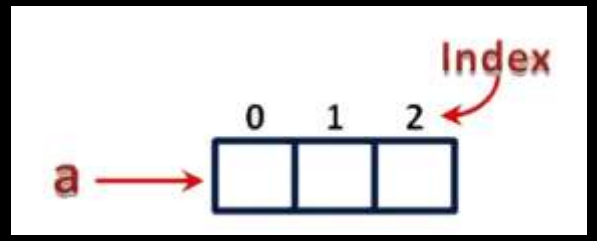
"Collection of similar data items stored in continuous memory locations with single name".

int a, b, c;

int a[3];

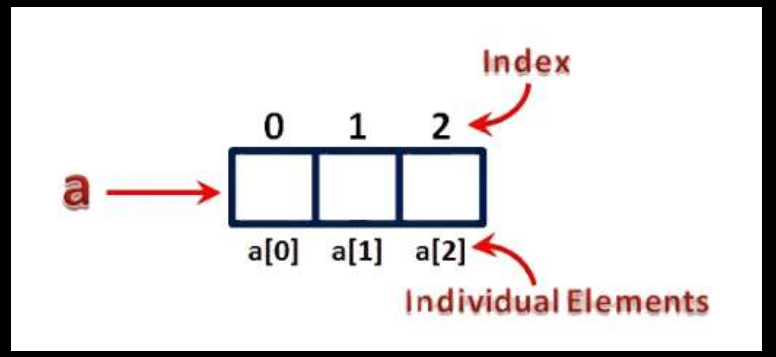


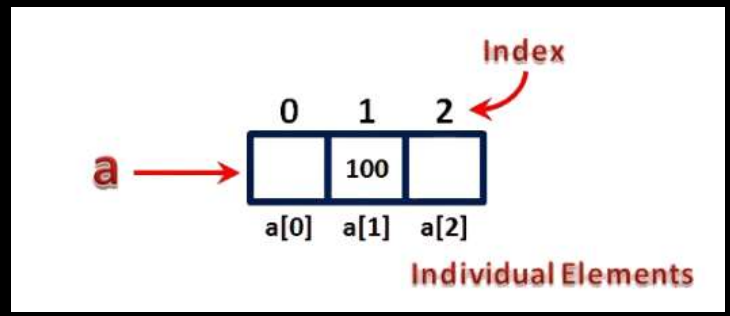




arrayName[indexValue]

a[1] = 100;





SPARSE MATRIX

Sparse matrix is a matrix which contains very few non-zero elements.

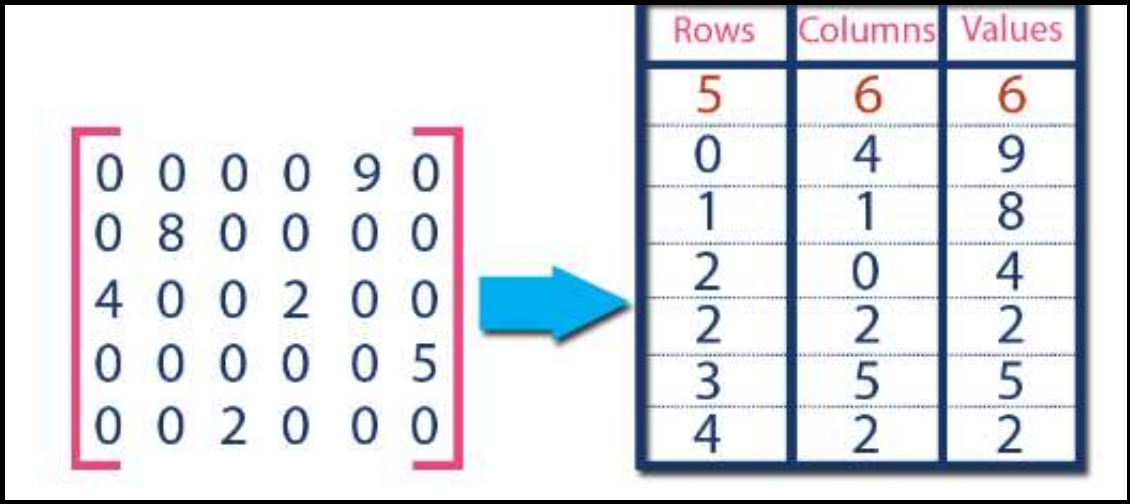
- **Sparse Matrix Representations**

Triplet Representation

Linked Representation

TRIPLET REPRESENTATION

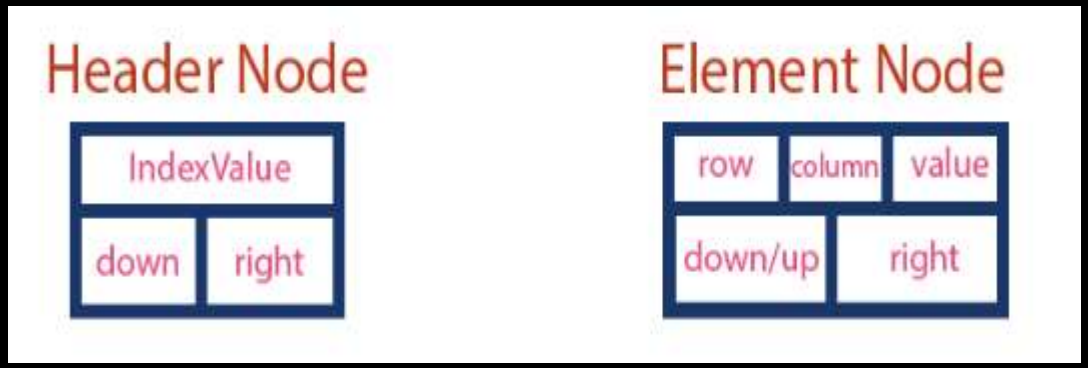
In this representation, we consider only non-zero values along with their row and column index values. In this representation, the 0th row stores total rows, total columns and total non-zero values in the matrix.

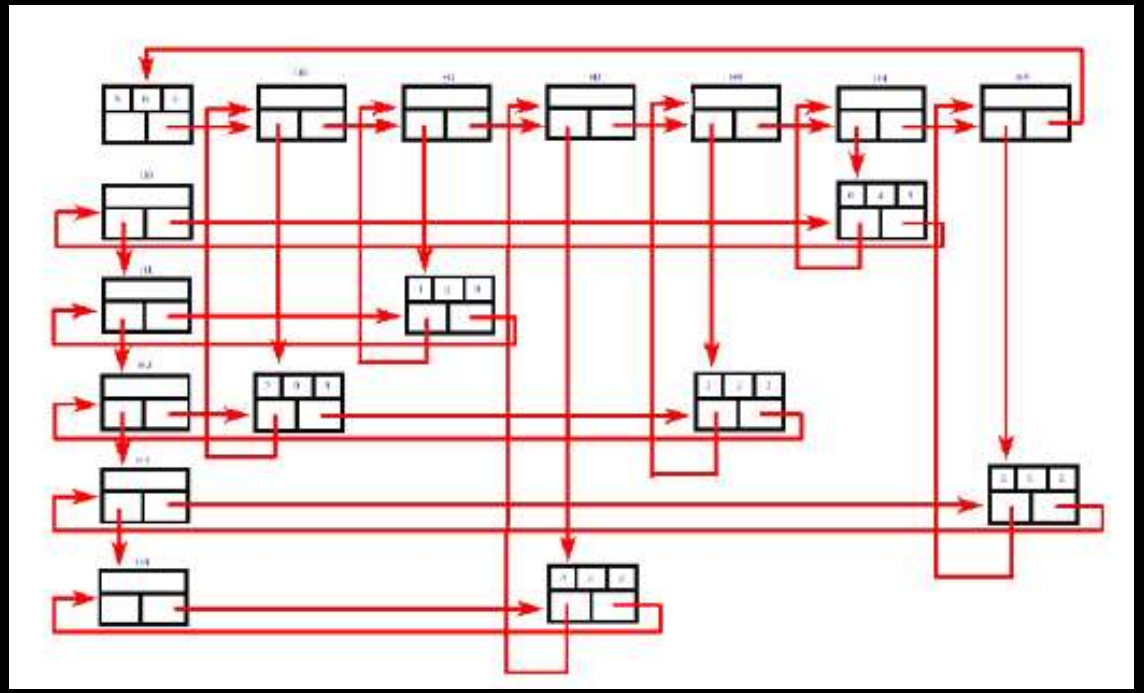


$$\begin{bmatrix} 0 & 0 & 0 & 0 & 9 & 0 \\ 0 & 8 & 0 & 0 & 0 & 0 \\ 4 & 0 & 0 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 5 \\ 0 & 0 & 2 & 0 & 0 & 0 \end{bmatrix}$$

Rows	Columns	Values
5	6	6
0	4	9
1	1	8
2	0	4
2	2	2
3	5	5
4	2	2

LINKED REPRESENTATION

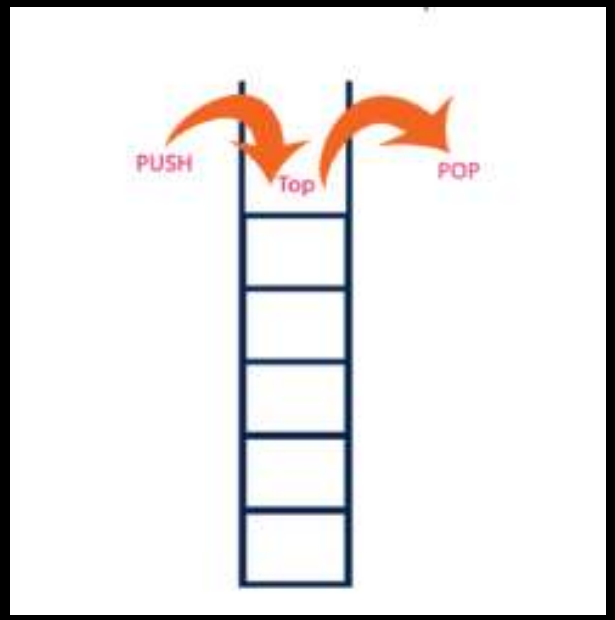




STACK-**LIFO** (LAST IN FIRST OUT)

Stack is a linear data structure in which the operations are performed based on **LIFO** principle.

"A Collection of similar data items in which both insertion and deletion operations are performed based on **LIFO** principle".



OPERATIONS ON A STACK

Push (To insert an element on to the stack)

Pop (To delete an element from the stack)

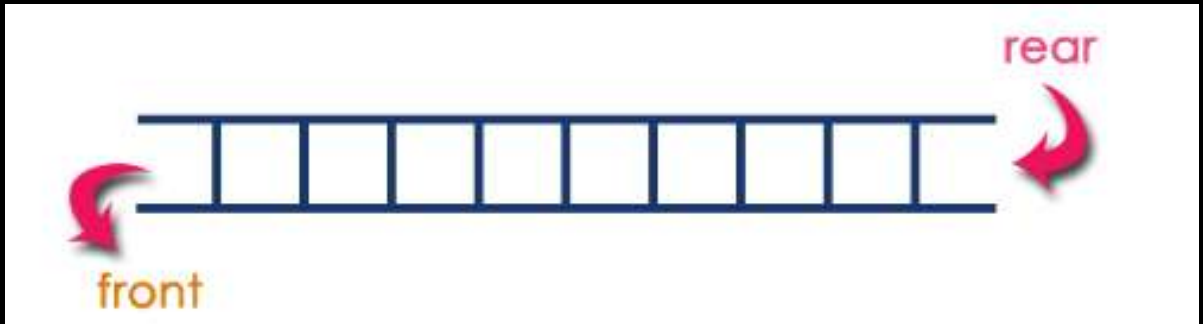
Display (To display elements of the stack)

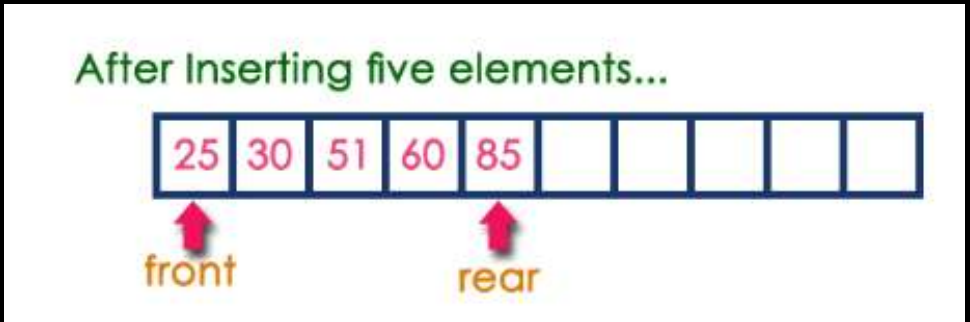
Stack data structure can be implement in two ways. They are as follows...

- **Using Array**
- **Using Linked List**

QUEUE-FIFO (FIRST IN FIRST OUT)

- Queue data structure is a linear data structure in which the operations are performed based on **FIFO** principle.
- "Queue data structure is a collection of similar data items in which insertion and deletion operations are performed based on **FIFO** principle".





OPERATIONS ON A QUEUE

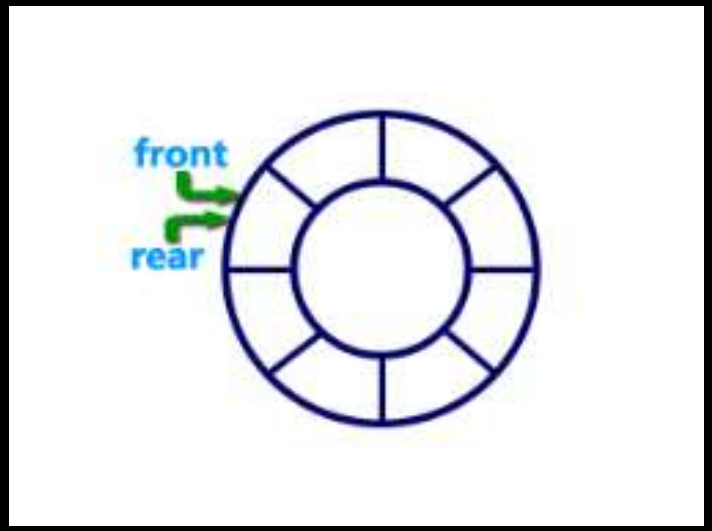
- **enQueue(value)** - (To insert an element into the queue)
- **deQueue()** - (To delete an element from the queue)
- **display()** - (To display the elements of the queue)

Queue data structure can be implemented in two ways. They are as follows...

- **Using Array**
- **Using Linked List**

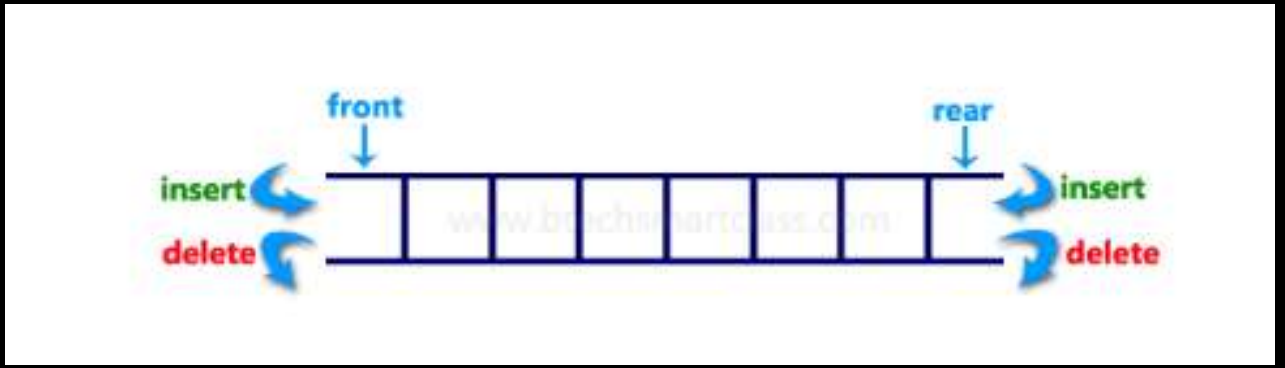
WHAT IS CIRCULAR QUEUE?

Circular Queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle.



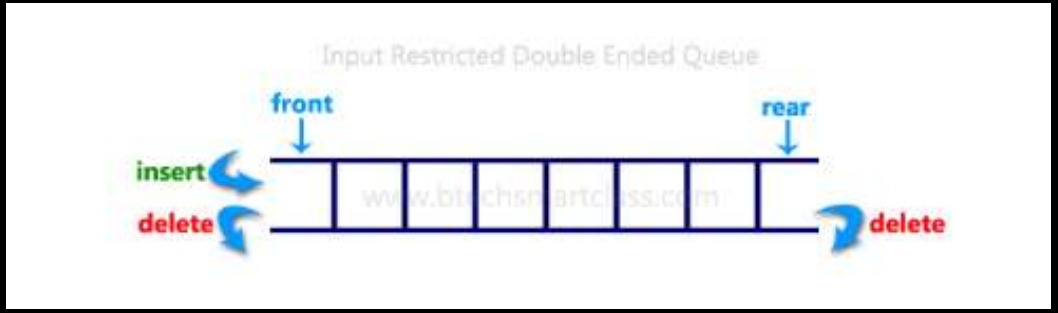
DOUBLE ENDED QUEUE (DEQUEUE)

- Double Ended Queue is also a Queue data structure in which the insertion and deletion operations are performed at both the ends (**front** and **rear**). That means, we can insert at both front and rear positions and can delete from both front and rear positions.



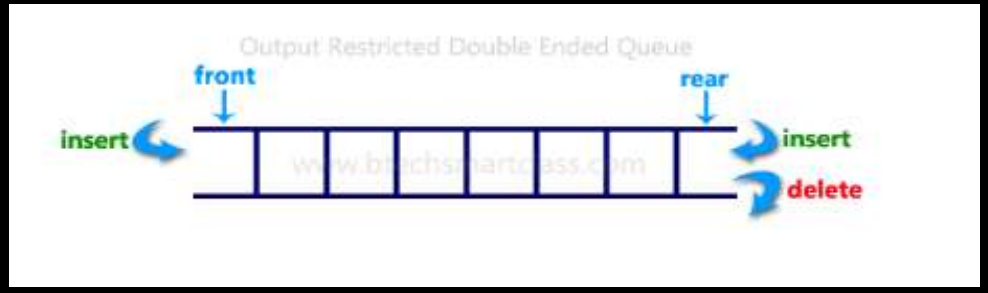
Input Restricted Double Ended Queue

In input restricted double ended queue, the insertion operation is performed at only one end and deletion operation is performed at both the ends.



Output Restricted Double Ended Queue

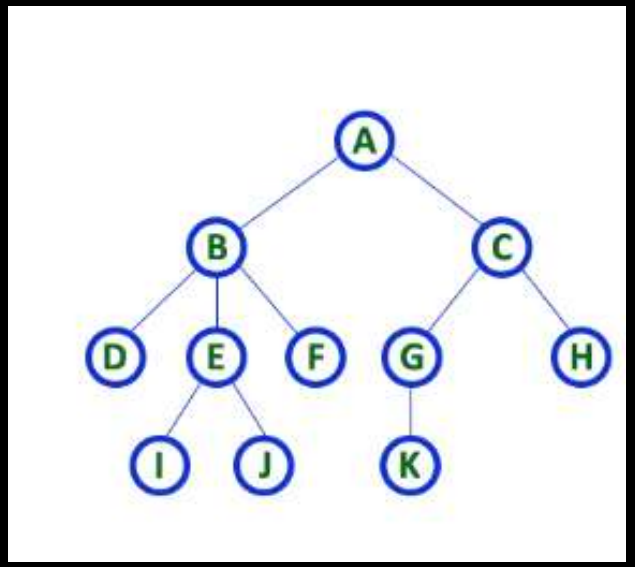
In output restricted double ended queue, the deletion operation is performed at only one end and insertion operation is performed at both the ends.



TREE

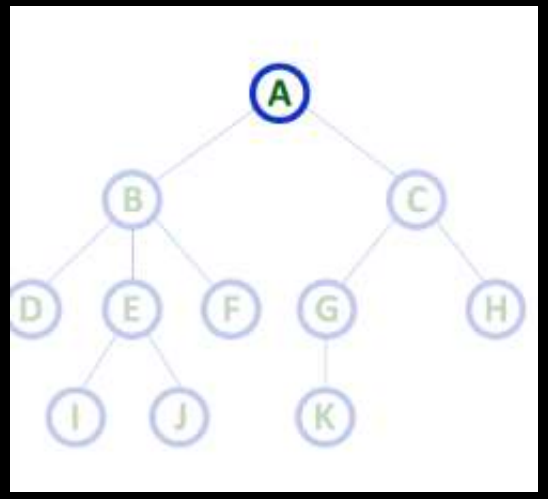
Tree is a non-linear data structure which organizes data in hierarchical structure and this is a recursive definition.

Tree data structure is a collection of data (Node) which is organized in hierarchical structure and this is a recursive definition

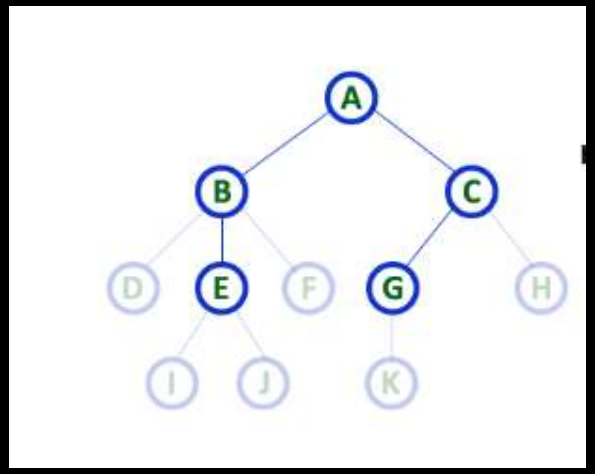


TERMINOLOGY

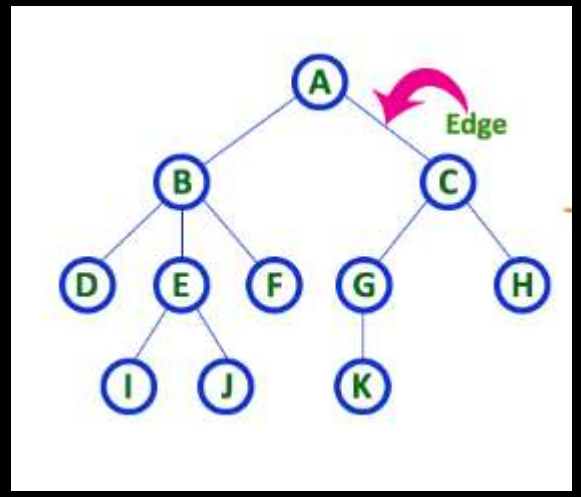
Root



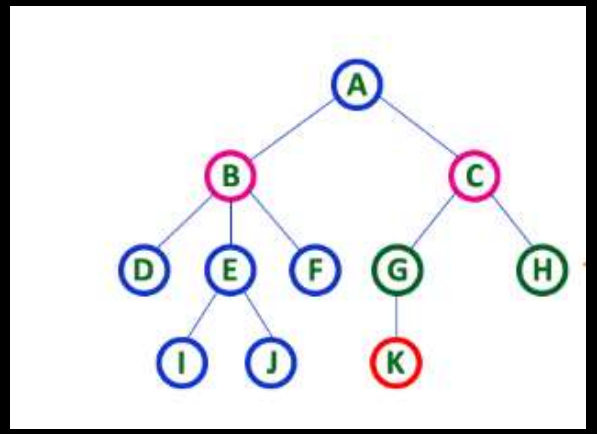
Parent



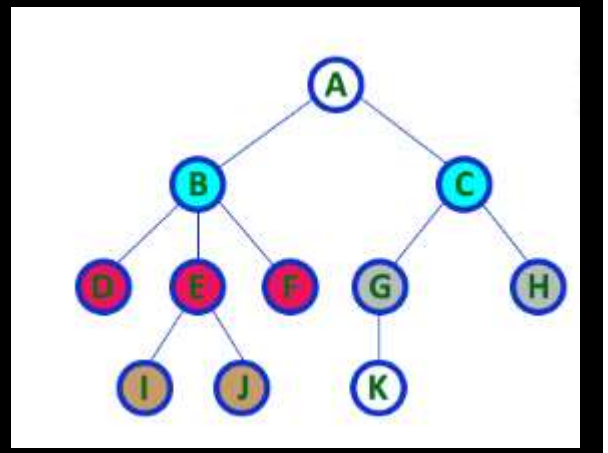
Edge



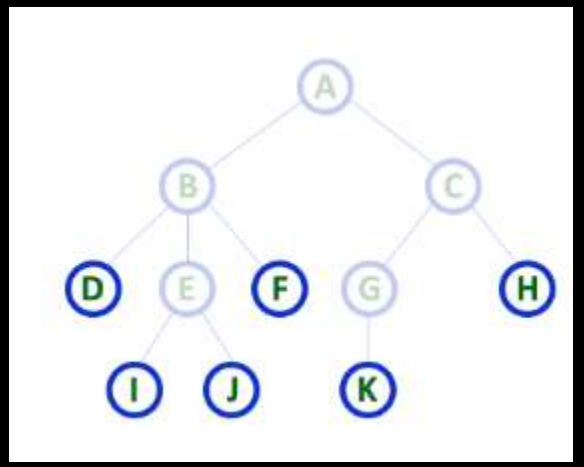
Child



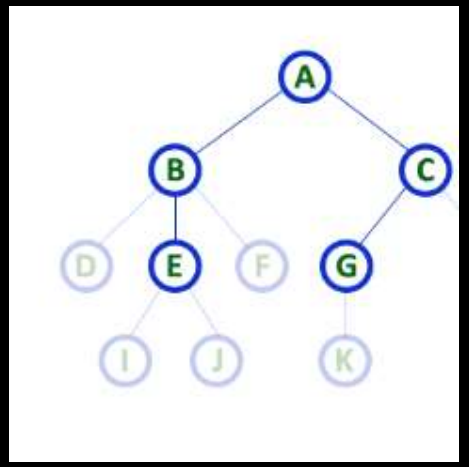
Siblings



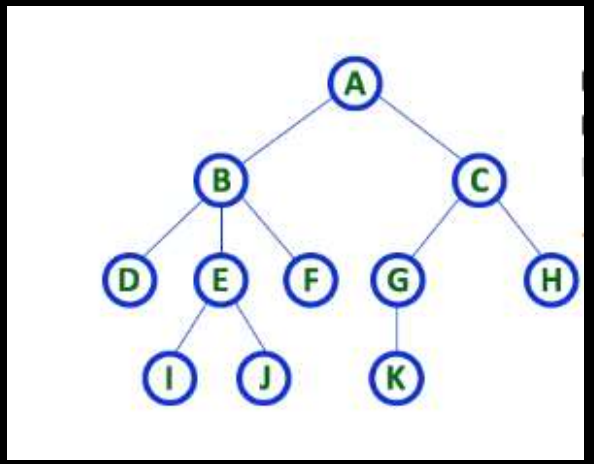
Leaf



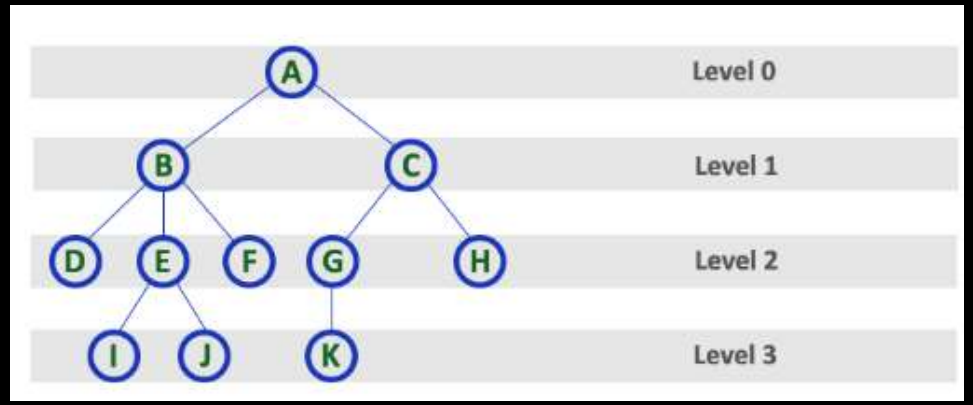
Internal Nodes



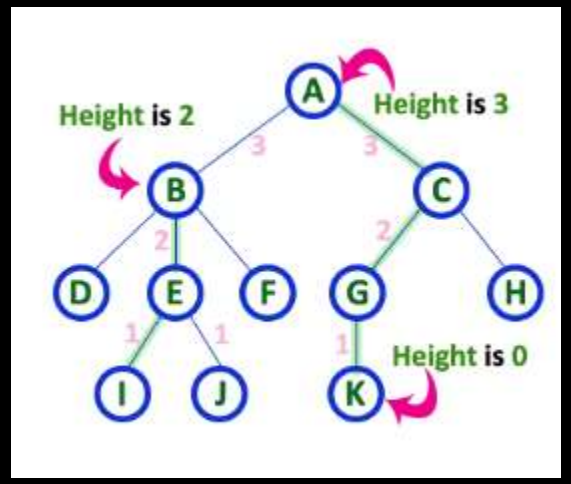
Degree



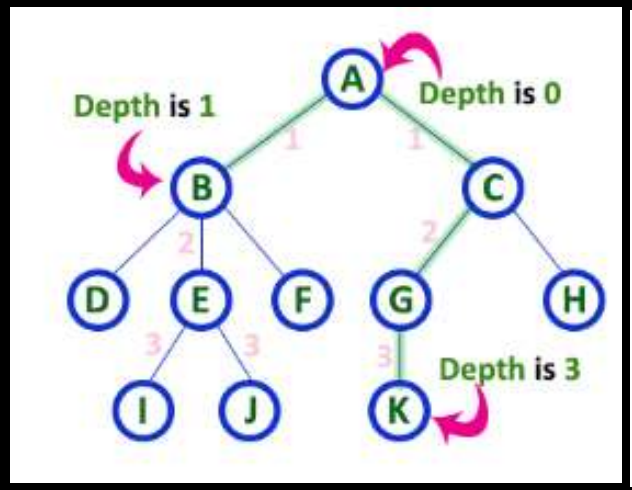
Level



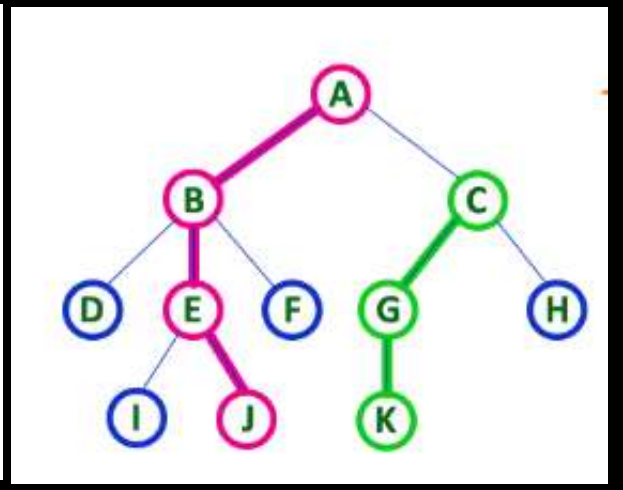
Height



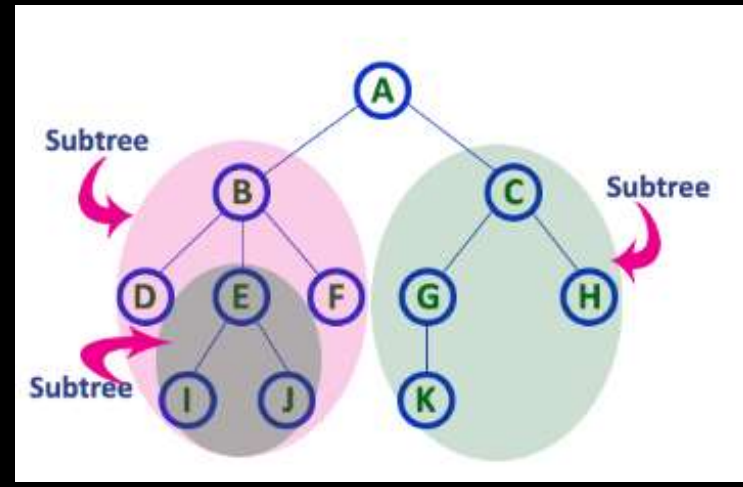
Depth



Path

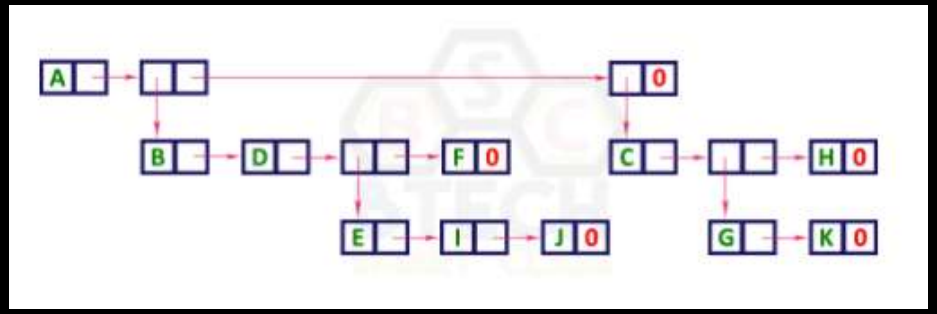


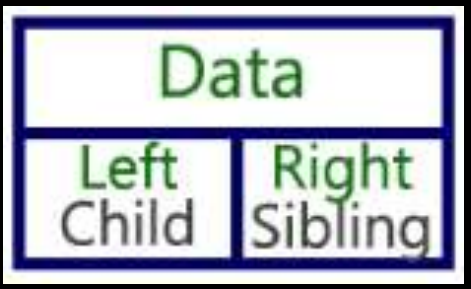
Sub Tree



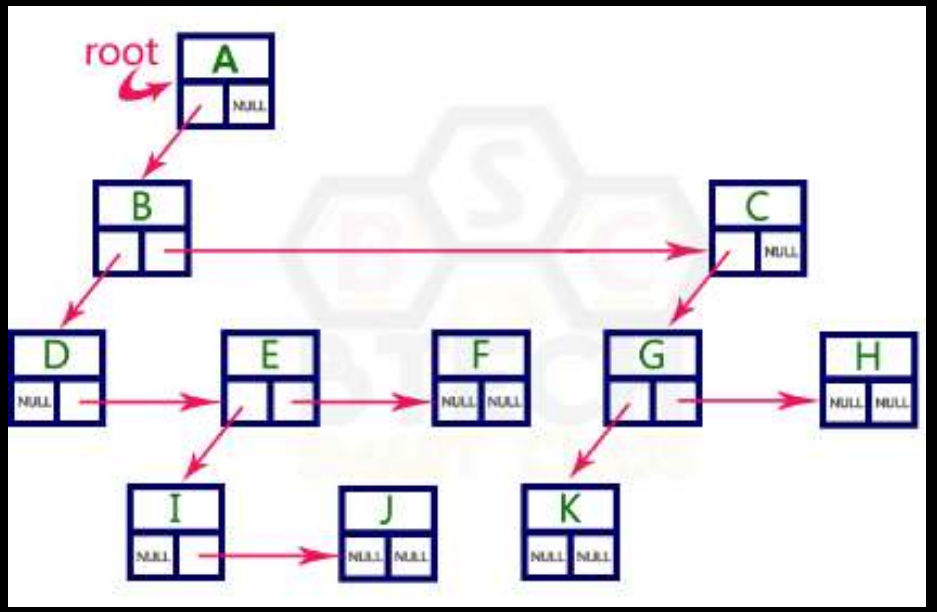
TREE REPRESENTATIONS

List Representation



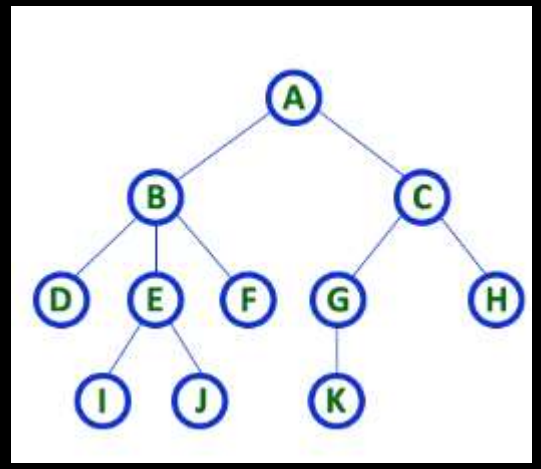


Left Child - Right Sibling Representation



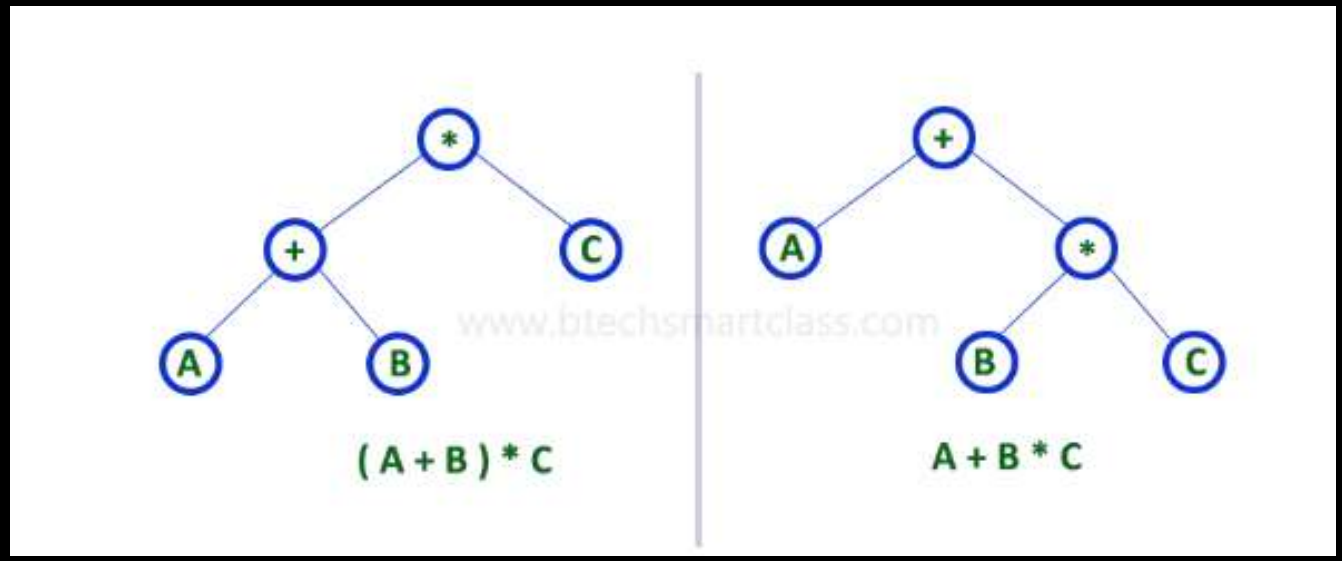
BINARY TREE

A tree in which every node can have a maximum of two children is called as Binary Tree.



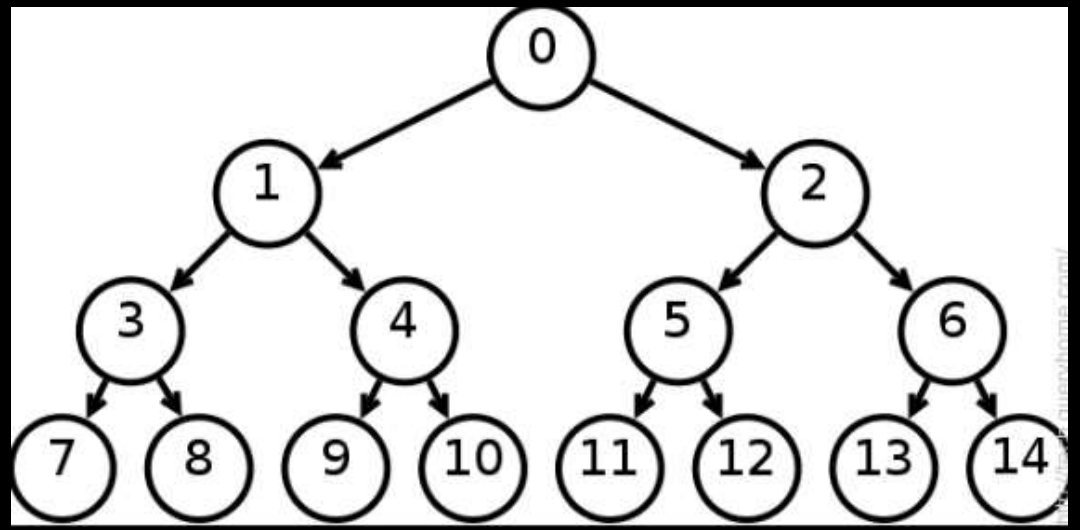
STRICTLY BINARY TREE

A binary tree in which every node has either two or zero number of children is called Strictly Binary Tree.



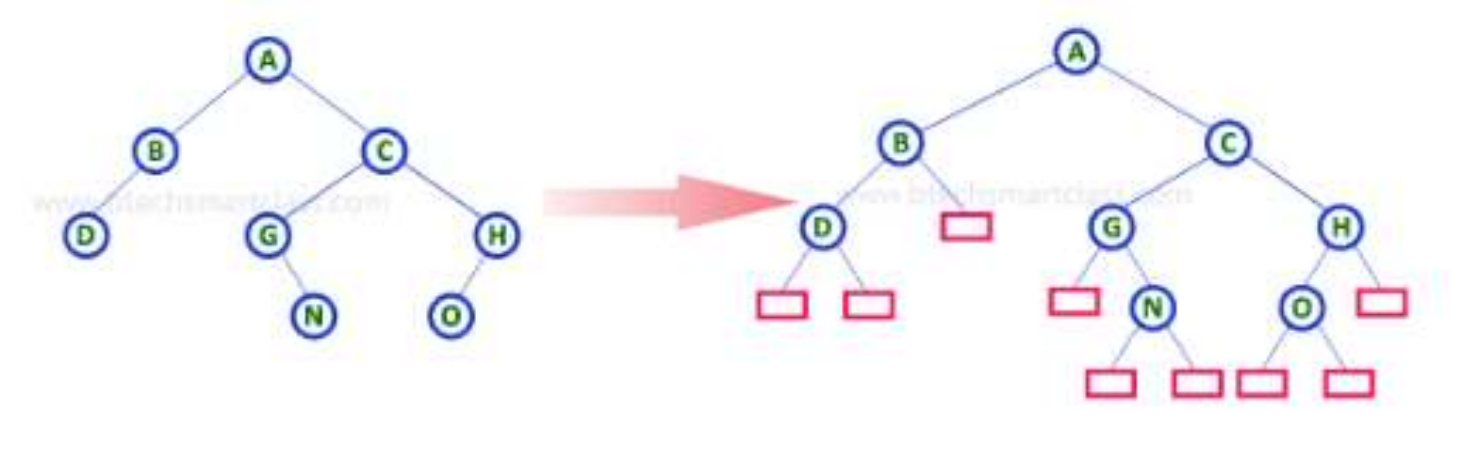
COMPLETE BINARY TREE

A binary tree in which every internal node has exactly two children and all leaf nodes are at same level is called Complete Binary Tree.



EXTENDED BINARY TREE

The full binary tree obtained by adding dummy nodes to a binary tree is called as Extended Binary Tree.



BINARY TREE TRAVERSALS

Displaying (or) visiting order of nodes in a binary tree is called as Binary Tree Traversal.

There are three types of binary tree traversals.

1. **In - Order Traversal**

I - D - J - B - F - A - G - K - C - H

2. **Pre - Order Traversal**

A - B - D - I - J - F - C - G - K - H

3. **Post - Order Traversal**

I - J - D - F - B - K - G - H - C - A

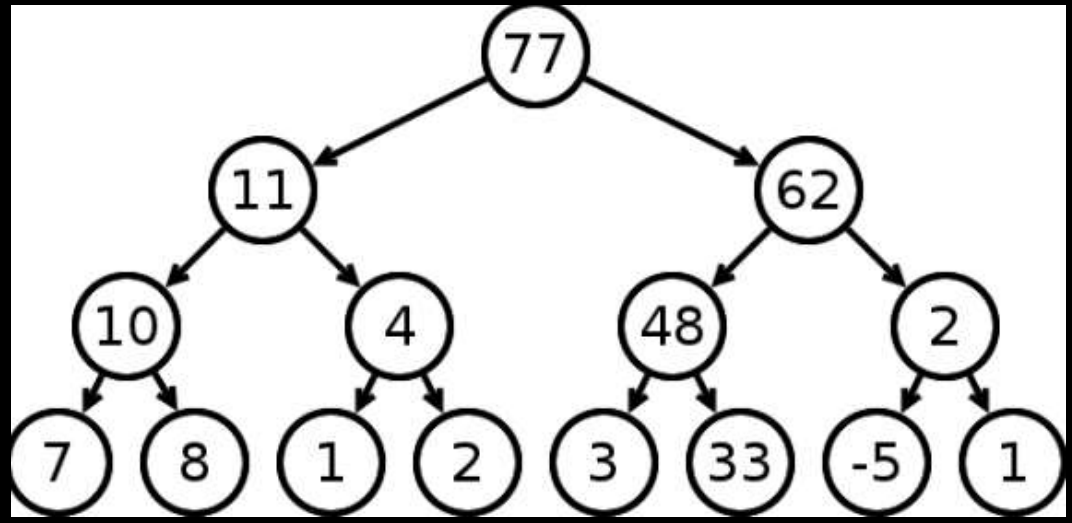
HEAP

Heap data structure is a specialized binary tree based data structure. Heap is a binary tree with special characteristics. In a heap data structure, nodes are arranged based on their value. A heap data structure, some time called as Binary Heap.

There are two types of heap data structures and they are as follows...

- **Max Heap**
- **Min Heap**

Max heap is a specialized full binary tree in which every parent node contains greater or equal value than its child nodes. And last leaf node can be alone.



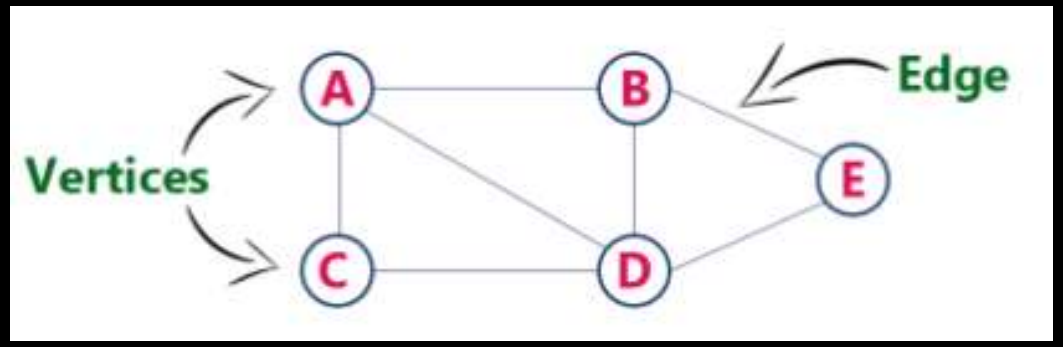
OPERATIONS ON MAX HEAP

The following operations are performed on a Max heap data structure...

- **Finding Maximum**
- **Insertion**
- **Deletion**

GRAPHS

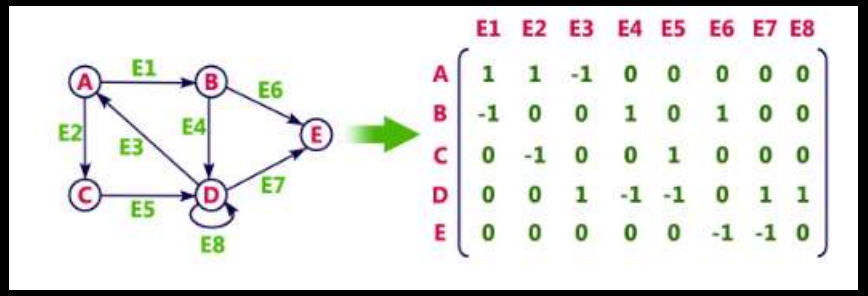
- **Graph is a collection of vertices and arcs which connects vertices in the graph**
- **Graph is a collection of nodes and edges which connects nodes in the graph**



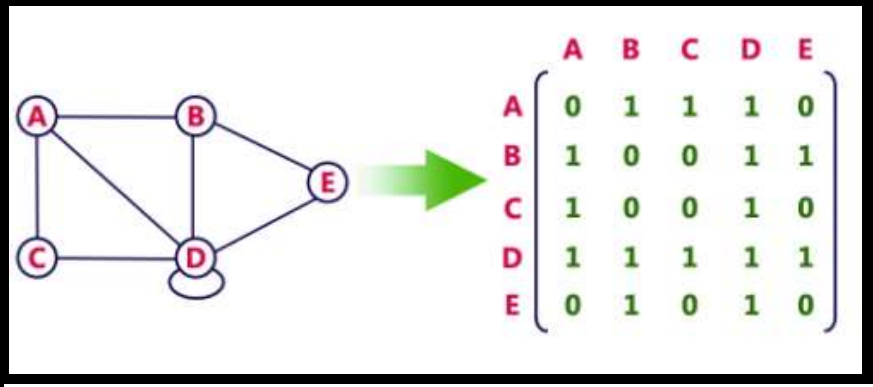
- Generally, a graph **G** is represented as **G = (V , E)**, where **V is set of vertices** and **E is set of edges**.

GRAPH REPRESENTATIONS

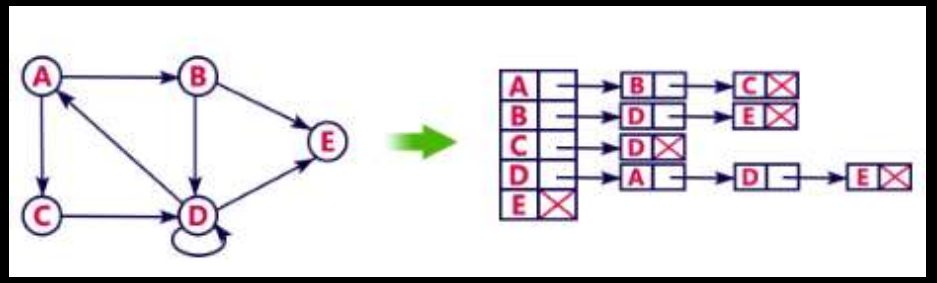
- Adjacency Matrix



- Incidence Matrix



- Adjacency List




THANK YOU!